Practical 2

Part A:

Aim:

1. Smart thermostat (Thermostat.java, main in this file)
(a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.

(c) Constructor(location): chain with this(location, 22).

(d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)".

(e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)".

(f) Add getTemperature() (no setter) and static getActiveCount().

(g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops

(printing the temperature each time), then print getActiveCount().
Expected output: temperature rises to 30 then "Already at maximum", falls to
16 then "Already at minimum",
count 2.

Code:

```
public class Thermostat {

    // Instance variables
private String location;
private int temperature;

    // Constants     private static
final int MIN = 16;     private static
final int MAX = 30;

    // Static variable     private
static int activeCount = 0;

    // Constructor with location and starting temperature
public Thermostat(String location, int startTemp) {
this.location = location;
         if (startTemp >= MIN && startTemp <= MAX)
{             this.temperature = startTemp;
        } else {
this.temperature = 22;
        }

activeCount++;
    }

    // Constructor with only location
public Thermostat(String location) {
this(location, 22);
    }

    // Increase temperature
```

```
    public void raise() {
if (temperature < MAX) {
temperature++;
        } else {
            System.out.println("Already at maximum (30)");
        }
    }

    // Decrease temperature
public void lower() {         if
(temperature > MIN) {
temperature--;
        } else {
            System.out.println("Already at minimum (16)");
        }
    }

    // Getter for temperature
public int getTemperature() {
return temperature;
    }

    // Static getter for active thermostat count
public static int getActiveCount() {
return activeCount;
    }

    // Main method     public static void
main(String[] args) {

        Thermostat t1 = new Thermostat("Living Room", 24);
        Thermostat t2 = new Thermostat("Bedroom");

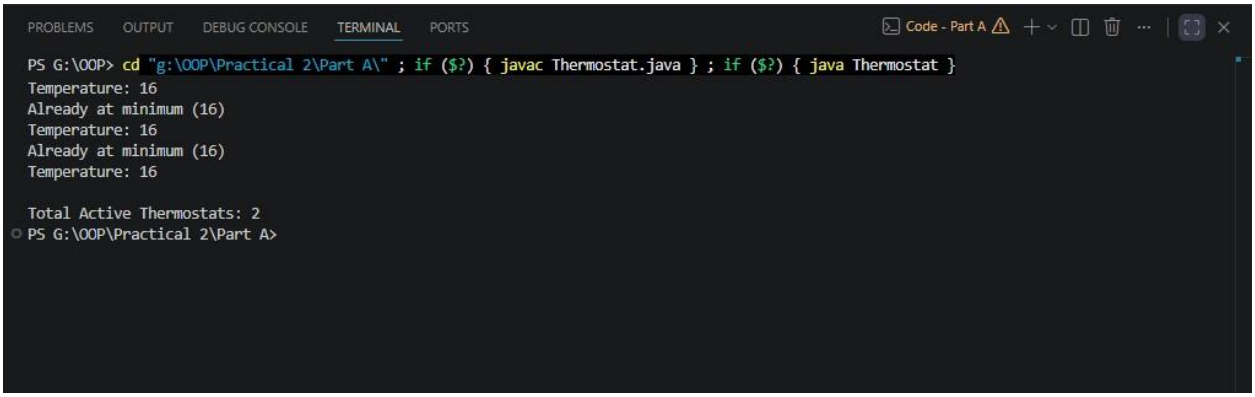
        System.out.println("Initial Temperature of Living Room:
" + t1.getTemperature());
```

```
        System.out.println("\nRaising temperature 10 times:");
for (int i = 1; i <= 10; i++) {
            t1.raise();
            System.out.println("Temperature: " +
t1.getTemperature());
        }

        System.out.println("\nLowering temperature 20 times:");
for (int i = 1; i <= 20; i++) {
            t1.lower();
            System.out.println("Temperature: " +
t1.getTemperature());
        }

        System.out.println("\nTotal Active Thermostats: " +
Thermostat.getActiveCount());
    } }
```

Output:



Aim:

2. Cinema show (CinemaShow.java, main in this file)

(a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0.

(b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100).

(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.

(d) cancel(int n): increase seatsAvailable by n but never above capacity.

(e) Add getSeatsAvailable() and static getTotalBooked().

(f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().

Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

Code:

```
public class CinemaShow {

    // Instance variables
private String title;     private
int seatsAvailable;     private
final int capacity;

    // Static variable     private
static int totalBooked = 0;

    // Constructor with title and capacity
public CinemaShow(String title, int capacity) {
this.title = title;         this.capacity =
capacity;         this.seatsAvailable = capacity;
    }

    // Constructor with only title
public CinemaShow(String title) {
this(title, 100);
    }

    // Book seats     public
boolean book(int n) {         if
(n <= seatsAvailable) {
seatsAvailable -= n;
totalBooked += n;
return true;
        } else {
            return false;
        }
    }
```

```
    // Cancel seats     public
void cancel(int n) {
seatsAvailable += n;
         if (seatsAvailable > capacity)
{             seatsAvailable = capacity;
        }
    }

    // Getter for available seats
public int getSeatsAvailable() {
return seatsAvailable;
    }

    // Static getter for total booked seats
public static int getTotalBooked() {
return totalBooked;
    }

    // Main method     public static void
main(String[] args) {

        CinemaShow show = new CinemaShow("Avengers", 50);
        System.out.println("Movie: Avengers");
System.out.println("Initial Seats: " +
show.getSeatsAvailable());

        System.out.println("\nBook 20 seats:");
System.out.println("Booking Successful: " + show.book(20));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        System.out.println("\nBook 15 seats:");
```

```
        System.out.println("Booking Successful: " +
show.book(15));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

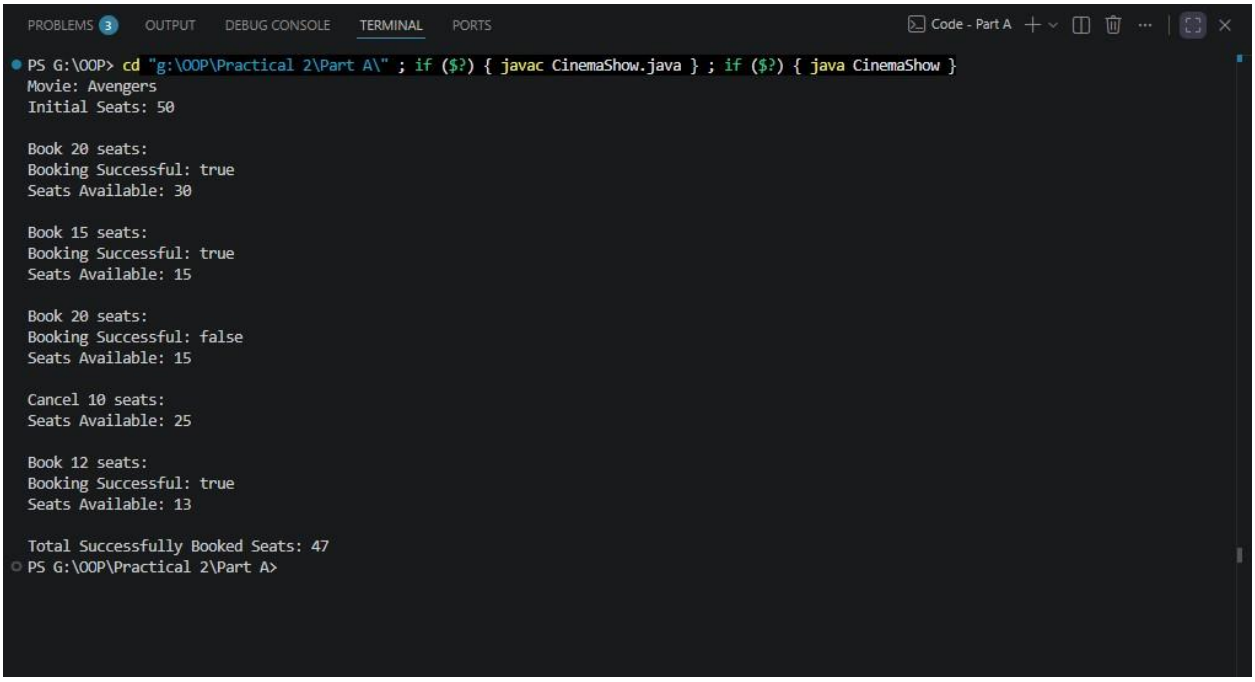
        System.out.println("\nBook 20 seats:");
System.out.println("Booking Successful: " + show.book(20));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

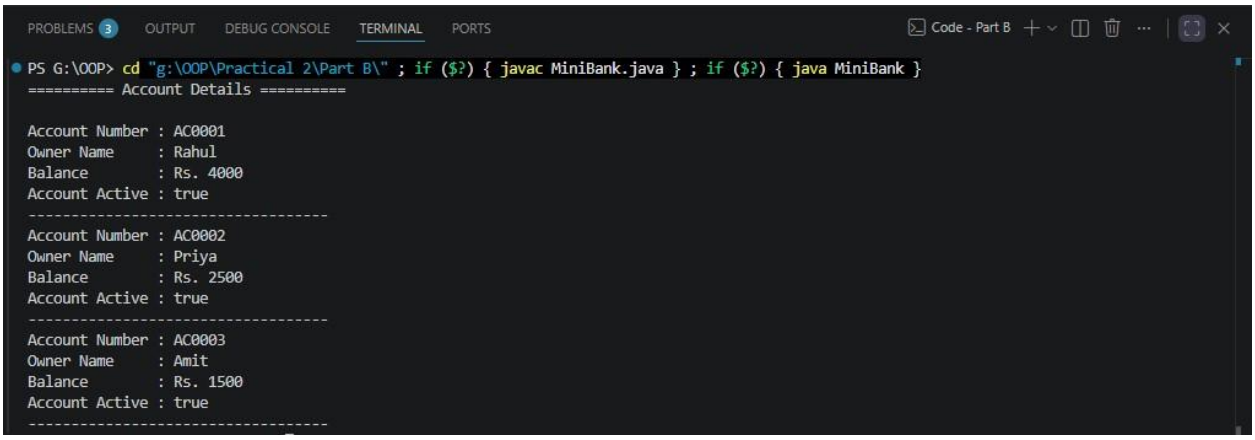
        System.out.println("\nCancel 10 seats:");
show.cancel(10);
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        System.out.println("\nBook 12 seats:");
System.out.println("Booking Successful: " + show.book(12));
        System.out.println("Seats Available: " +
show.getSeatsAvailable());

        System.out.println("\nTotal Successfully Booked Seats:
" + CinemaShow.getTotalBooked());
    } }
```

Output:


PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 2\Part A\" ; if ($?) { javac CinemaShow.java } ; if ($?) { java CinemaShow }
Movie: Avengers
Initial Seats: 50
Book 20 seats:
Booking Successful: true
Seats Available: 30
Book 15 seats:
Booking Successful: true
Seats Available: 15
Book 20 seats:
Booking Successful: false
Seats Available: 15
Cancel 10 seats:
Seats Available: 25
Book 12 seats:
Booking Successful: true
Seats Available: 13
Total Successfully Booked Seats: 47
PS G:\OOP\Practical 2\Part A>


PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Code - Part B
PS G:\OOP> cd "g:\OOP\Practical 2\Part B\" ; if ($?) { javac MiniBank.java } ; if ($?) { java MiniBank }
========== Account Details ==========
Account Number : AC0001
Owner Name : Rahul
Balance : Rs. 4000
Account Active : true

Account Number : AC0002
Owner Name : Priya
Balance : Rs. 2500
Account Active : true

Account Number : AC0003
Owner Name : Amit
Balance : Rs. 1500
Account Active : true
---

3. (For Advanced Learners – home) Parking lot (ParkingLot.java)
(a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0.
(b) park(String type): for "two", if twoWheelers < twoCap increment and add 20 to revenue; for "four", if fourWheelers < fourCap increment and add 40; otherwise print "Full" and reject.
(c) leave(String type): decrement the matching count but never below 0.
(d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.
Expected output: parking into a full section is refused; revenue = 20×(two parked) + 40×(four parked).

Code:

ParkingLot.java

```
public class ParkingLot {

    // Current occupancy
    private int twoWheelers;
    private int fourWheelers;

    // Maximum capacity
    private final int twoCap;
    private final int fourCap;

    // Total revenue
    private static long revenue = 0;

    // Constructor
    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
        this.twoWheelers = 0;
        this.fourWheelers = 0;
    }

    // Park a vehicle
    public void park(String type) {

        if (type.equalsIgnoreCase("two")) {

            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
```

```
                System.out.println("Two-wheeler parked
successfully.");
            } else {
                System.out.println("Full");
            }

        } else if (type.equalsIgnoreCase("four")) {

            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                System.out.println("Four-wheeler parked
successfully.");
            } else {
                System.out.println("Full");
            }

        } else {
            System.out.println("Invalid vehicle type.");
        }
    }

    // Vehicle leaves
    public void leave(String type) {

        if (type.equalsIgnoreCase("two")) {

            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two-wheeler left.");
            } else {
                System.out.println("No two-wheeler to leave.");
            }

        } else if (type.equalsIgnoreCase("four")) {
```

```
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four-wheeler left.");
            } else {
                System.out.println("No four-wheeler to leave.");
            }

        } else {
            System.out.println("Invalid vehicle type.");
        }
    }

    // Display final status
    public void displayStatus() {
        System.out.println("\n=== Parking Lot Status ===");
        System.out.println("Two-wheelers : " + twoWheelers + "/"
+ twoCap);
        System.out.println("Four-wheelers: " + fourWheelers +
"/" + fourCap);
        System.out.println("Revenue      : " + revenue);
    }

    public static void main(String[] args) {

        // Capacity: 2 two-wheelers and 2 four-wheelers
        ParkingLot lot = new ParkingLot(2, 2);

        System.out.println("=== Parking Events ===");

        // Two-wheelers
        lot.park("two");
        lot.park("two");
```

```
        // This should be refused because two-wheeler section is
full
        lot.park("two");

        // Four-wheelers
        lot.park("four");
        lot.park("four");

        // This should be refused because four-wheeler section
is full
        lot.park("four");

        // Leave events
        lot.leave("two");
        lot.leave("four");

        // Park again after space becomes available
        lot.park("two");
        lot.park("four");

        // Display final result
        lot.displayStatus();
    }
}
```

Output


Problems 6
Output
Debug Console
Terminal
Ports
Code - Part A
PS G:\OOP> cd "g:\OOP\Practical 2\Part A\" ; if ($?) { javac ParkingLot.java } ; if ($?) { java ParkingLot }
=== Parking Events ===
Two-wheeler parked successfully.
Two-wheeler parked successfully.
Full
Four-wheeler parked successfully.
Four-wheeler parked successfully.
Full
Two-wheeler left.
Four-wheeler left.
Two-wheeler parked successfully.
Four-wheeler parked successfully.
=== Parking Lot Status ===
Two-wheelers : 2/2
Four-wheelers: 2/2
Revenue : 180
PS G:\OOP\Practical 2\Part A>

Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is encapsulation, and how do private fields with public getters enforce it?

Ans.

Encapsulation means keeping an object's data protected inside the class and controlling how that data can be accessed or changed.

For example:

```
private long balance;

public long getBalance() {
    return balance;
}
```

Because `balance` is `private`, code outside the `Account` class cannot directly do:

```
account.balance = 10000;    // ✗ Not allowed
```

It must use the methods provided by the class:

```
account.getBalance();       // ✓
account.deposit(1000);      // ✓
account.withdraw(500);      // ✓
```

This allows the class to **control and validate changes** to its data.

2. Why is accountNumber declared final?

Ans.

`accountNumber` is declared:

```
private final String accountNumber;
```

because an account number should be **assigned once when the account is created and should not change afterward**.

For example:

```
public Account(String ownerName, long openingBalance) {
    this.accountNumber = generateAccountNumber();
}
```

After that, this is not allowed:

```
accountNumber = "AC9999";   // ✕ Cannot assign a value
to final variable
```

This protects the identity of the account.

3. What is the difference between a static field and an instance field when generating ids?

Ans.

Static field

A `static` field belongs to the **class**, so there is **one shared copy** for all objects.

Example:

```
private static int accountCounter = 1;
```

If we create three accounts:

```
Account 1 → AC0001
Account 2 → AC0002
Account 3 → AC0003
```

The same `accountCounter` is shared by all three accounts.

Instance field

An instance field belongs to **each individual object**.

For example:

```
private String accountNumber;
```

Every `Account` object has its own `accountNumber`.

```
Account 1 → accountNumber = AC0001
Account 2 → accountNumber = AC0002
Account 3 → accountNumber = AC0003
```